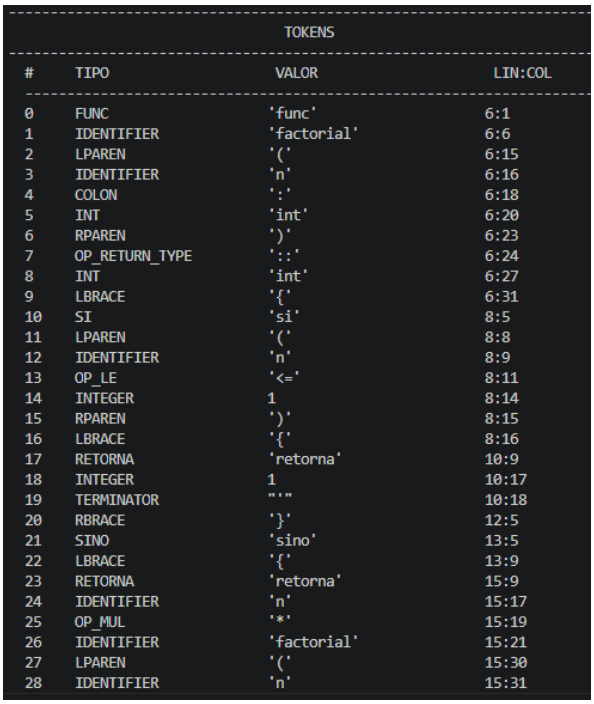
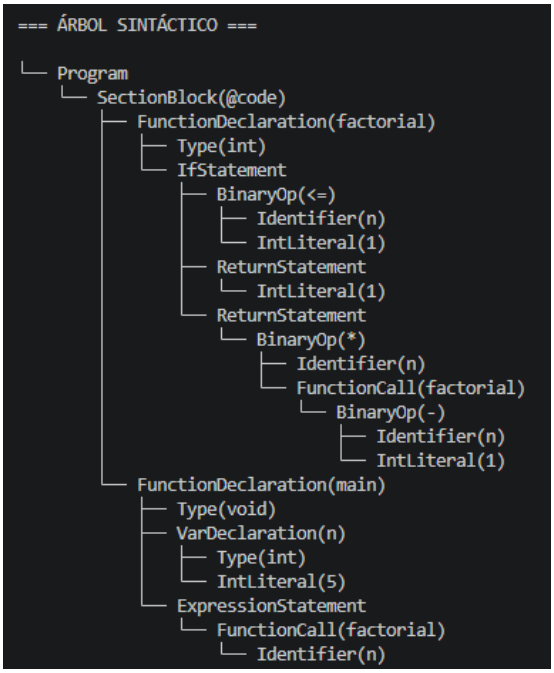
Evidencias de ejecución para los diferentes programas

Ejercicio 1. Factorial



TOKENS

#	TIPO	VALOR	LIN:COL
0	FUNC	'func'	6:1
1	IDENTIFIER	'factorial'	6:6
2	LPAREN	'('	6:15
3	IDENTIFIER	'n'	6:16
4	COLON	':'	6:18
5	INT	'int'	6:20
6	RPAREN	')'	6:23
7	OP_RETURN_TYPE	'::'	6:24
8	INT	'int'	6:27
9	LBRACE	'{'	6:31
10	SI	'si'	8:5
11	LPAREN	'('	8:8
12	IDENTIFIER	'n'	8:9
13	OP_LE	'<='	8:11
14	INTEGER	1	8:14
15	RPAREN	')'	8:15
16	LBRACE	'{'	8:16
17	RETORNA	'retorna'	10:9
18	INTEGER	1	10:17
19	TERMINATOR	"'"	10:18
20	RBRACE	'}'	12:5
21	SINO	'sino'	13:5
22	LBRACE	'{'	13:9
23	RETORNA	'retorna'	15:9
24	IDENTIFIER	'n'	15:17
25	OP_MUL	'*'	15:19
26	IDENTIFIER	'factorial'	15:21
27	LPAREN	'('	15:30
28	IDENTIFIER	'n'	15:31



```
=== ÁRBOL SINTÁCTICO ===

└── Program
    └── SectionBlock(@code)
        ├── FunctionDeclaration(factorial)
        │   ├── Type(int)
        │   └── IfStatement
        │       ├── BinaryOp(<=)
        │       │   ├── Identifier(n)
        │       │   └── IntLiteral(1)
        │       ├── ReturnStatement
        │       │   └── IntLiteral(1)
        │       └── ReturnStatement
        │           └── BinaryOp(*)
        │               ├── Identifier(n)
        │               └── FunctionCall(factorial)
        │                   └── BinaryOp(-)
        │                       ├── Identifier(n)
        │                       └── IntLiteral(1)
        └── FunctionDeclaration(main)
            ├── Type(void)
            ├── VarDeclaration(n)
            │   ├── Type(int)
            │   └── IntLiteral(5)
            └── ExpressionStatement
                └── FunctionCall(factorial)
                    └── Identifier(n)
```

```
TABLA DE SIMBOLOS
--------------------------------------------------------------------------
  NOMBRE              KIND     TIPO              AMBITO          DIRECCIO

ambito: global
  factorial           func     (n:int) -> int          global          N/A
  main                func     () -> void        global          N/A

ambito: factorial (local)
  n                   param    int               factorial       0x0000
  else_0              label    label             factorial       N/A
  endif_1             label    label             factorial       N/A

ambito: main (local)
  n                   var      int               main            0x0000


               ETIQUETAS DE SALTO
--------------------------------------------------
  else_0                        se resuelve en la otra etapa
  endif_1                       se resuelve en la otra etapa


                    MAPA DE MEMORIA
  NOMBRE              TIPO              DIRECCION    AMBITO
--------------------------------------------------------------------
  n                   int               0x0000   local (factorial)
  n                   int               0x0000   local (main)
```

```
=== ASM GENERADO ===

lli r2, 0, 0
lli r2, 0xE0, 1

# --- inicialización de variables globales ---:

jal r1, main
halt

# --- funciones ---:
factorial:
add r8, r4, r0
addi r9, r0, 1
bgt r8, r9, else_0
addi r9, r0, 1
add r4, r9, r0
jr r1
jal r0, endif_1
else_0:
add r9, r4, r0
addi r2, r2, -4
store r9, 0(r2)
addi r2, r2, -4
store r1, 0(r2)
add r9, r4, r0
addi r8, r0, 1
sub  r10, r9, r8
add r4, r10, r0
jal r1, factorial
load r1, 0(r2)
addi r2, r2, 4
add r10, r4, r0
load r8, 0(r2)
```

```
=== ASM RESUELTO ===

lli r2, 0, 0
lli r2, 0xE0, 1
jal r1, 34
halt
add r8, r4, r0
addi r9, r0, 1
bgt r8, r9, 5
addi r9, r0, 1
add r4, r9, r0
jr r1
jal r0, 25
add r9, r4, r0
addi r2, r2, -4
store r9, 0(r2)
addi r2, r2, -4
store r1, 0(r2)
add r9, r4, r0
addi r8, r0, 1
sub  r10, r9, r8
add r4, r10, r0
jal r1, -16
load r1, 0(r2)
addi r2, r2, 4
add r10, r4, r0
load r8, 0(r2)
addi r2, r2, 4
addi r9, r0, 0
add  r11, r10, r0
```

```
=== TABLA DE ETIQUETAS (Resolver) ===
  factorial                      0x0010
  else_0                         0x002C
  mul_loop_0                     0x0074
  mul_end_0                      0x0084
  endif_1                        0x008C
  main                           0x0090
Binario generado : .\Codigos\factorial.bin  (49 instrucciones, 196 bytes)
Hex dump generado: .\Codigos\factorial.mem
```

factorial.mem

Codigos > factorial.mem

```
110000 // lli r2, 0, 0
1102E0 // lli r2, 0xE0, 1
388022 // jal r1, 34
480000 // halt
642000 // add r8, r4, r0
0C8001 // addi r9, r0, 1
34C405 // bgt r8, r9, 5
0C8001 // addi r9, r0, 1
624800 // add r4, r9, r0
408000 // jr r1
380019 // jal r0, 25
64A000 // add r9, r4, r0
0917FC // addi r2, r2, -4
249000 // store r9, 0(r2)
0917FC // addi r2, r2, -4
209000 // store r1, 0(r2)
64A000 // add r9, r4, r0
0C0001 // addi r8, r0, 1
654C10 // sub  r10, r9, r8
625000 // add r4, r10, r0
38FFF0 // jal r1, -16
189000 // load r1, 0(r2)
091004 // addi r2, r2, 4
652000 // add r10, r4, r0
1C1000 // load r8, 0(r2)
091004 // addi r2, r2, 4
0C8000 // addi r9, r0, 0
65D000 // add  r11, r10, r0
0D0001 // addi r10, r0, 1
285804 // beq  r11, r0, 4
64CC00 // add  r9, r9, r8
65DD10 // sub  r11, r11, r10
387FFD // jal  r0, -3
624800 // add r4, r9, r0
408000 // jr r1
408000 // jr r1
0917FC // addi r2, r2, -4
0C8005 // addi r9, r0, 5
249000 // store r9, 0(r2)
0917FC // addi r2, r2, -4
209000 // store r1, 0(r2)
1C9004 // load r9, 4(r2)
```

Ejercicio 2. Busqueda

busqueda.yeison

Codigos > busqueda.yeison

```
1
2   @code
3   func buscar(datos : [5]int, objetivo : int) :: int {
4       var i      : int = 0'
5       var encontrado : int = 0'
6       var posicion   : int = 0'
7
8       mientras (i < 5) {
9           si (datos[i] == objetivo) {
10              encontrado = 1'
11              posicion   = i'
12              i = 5'          # forzar salida del ciclo
13          } sino {
14              i = i + 1'
15          }
16      }
17
18      si (encontrado == 1) {
19          retorna posicion'
20      } sino {
21          retorna 0'          # no encontrado
22      }
23  }
24  func main() :: int {
25      var datos : [5]int = [10, 25, 37, 42, 8]'
26      var objetivo : int = 37'
27
28      var pos : int = buscar(datos, objetivo)'
29      retorna pos'
30  }
31
```

```
piler-feature-bin\custom-binary-compiler-feature-bin\main.py .\Codigos\busqueda.yeison -a
--------------------------------------------------------------------------
                              TOKENS
--------------------------------------------------------------------------
  #    TIPO                 VALOR                LIN:COL
--------------------------------------------------------------------------
  0    ANNOTATION           '@code'              2:1
  1    FUNC                 'func'               3:1
  2    IDENTIFIER           'buscar'             3:6
  3    LPAREN               '('                  3:12
  4    IDENTIFIER           'datos'              3:13
  5    COLON                ':'                  3:19
  6    LBRACKET             '['                  3:21
  7    INTEGER              5                    3:22
  8    RBRACKET             ']'                  3:23
  9    INT                  'int'                3:24
  10   COMMA                ','                  3:27
  11   IDENTIFIER           'objetivo'           3:29
  12   COLON                ':'                  3:38
  13   INT                  'int'                3:40
  14   RPAREN               ')'                  3:43
  15   OP_RETURN_TYPE       '::'                 3:45
  16   INT                  'int'                3:48
  17   LBRACE               '{'                  3:52
  18   VAR                  'var'                4:5
  19   IDENTIFIER           'i'                  4:9
  20   COLON                ':'                  4:16
  21   INT                  'int'                4:18
  22   ASSIGN               '='                  4:22
  23   INTEGER              0                    4:24
  24   TERMINATOR           "'"                  4:25
  25   VAR                  'var'                5:5
  26   IDENTIFIER           'encontrado'         5:9
  27   COLON                ':'                  5:20
  28   INT                  'int'                5:22
```

```
piler-feature-bin\custom-binary-compiler-feature-bin\main.py .\Codigos\busqueda.yeison -a
=== ÁRBOL SINTÁCTICO ===

└── Program
    └── SectionBlock(@code)
        ├── FunctionDeclaration(buscar)
        │   ├── Type(int)
        │   ├── VarDeclaration(i)
        │   │   ├── Type(int)
        │   │   └── IntLiteral(0)
        │   ├── VarDeclaration(encontrado)
        │   │   ├── Type(int)
        │   │   └── IntLiteral(0)
        │   ├── VarDeclaration(posicion)
        │   │   ├── Type(int)
        │   │   └── IntLiteral(0)
        │   ├── WhileStatement
        │   │   ├── BinaryOp(<)
        │   │   │   ├── Identifier(i)
        │   │   │   └── IntLiteral(5)
        │   │   └── IfStatement
        │   │       ├── BinaryOp(==)
        │   │       │   ├── IndexAccess
        │   │       │   │   ├── Identifier(datos)
        │   │       │   │   └── Identifier(i)
        │   │       │   └── Identifier(objetivo)
        │   │       ├── Assignment
        │   │       │   ├── Identifier(encontrado)
        │   │       │   └── IntLiteral(1)
        │   │       ├── Assignment
        │   │       │   ├── Identifier(posicion)
        │   │       │   └── Identifier(i)
        │   │       ├── Assignment
        │   │       │   ├── Identifier(i)
        │   │       │   └── IntLiteral(5)
        │   │       └── Assignment
        │   │           ├── Identifier(i)
        │   │           └── BinaryOp(+)
        │   │               ├── Identifier(i)
        │   │               └── IntLiteral(1)
        │   └── IfStatement
        │       ├── BinaryOp(==)
        │       │   ├── Identifier(encontrado)
        │       │   └── IntLiteral(1)
        │       ├── ReturnStatement
        │       │   └── Identifier(posicion)
        │       └── ReturnStatement
        │           └── IntLiteral(0)
        └── FunctionDeclaration(main)
            ├── Type(int)
```

```
TABLA DE SIMBOLOS
------------------------------------------------------------------------
  NOMBRE              KIND     TIPO              AMBITO          DIRECCION

ambito: global
  buscar              func     (datos:[5]int, objetivo:int) -> int          global          N/A
  main                func     () -> int         global          N/A

ambito: buscar (local)
  datos               param    [5]int            buscar          0x0000
  objetivo            param    int               buscar          0x0014
  i                   var      int               buscar          0x0018
  encontrado          var      int               buscar          0x001C
  posicion            var      int               buscar          0x0020
  while_start_0       label    label             buscar          N/A
  while_end_1         label    label             buscar          N/A
  else_2              label    label             buscar          N/A
  endif_3             label    label             buscar          N/A
  else_4              label    label             buscar          N/A
  endif_5             label    label             buscar          N/A

ambito: main (local)
  datos               var      [5]int            main            0x0000
  objetivo            var      int               main            0x0014
  pos                 var      int               main            0x0018


               ETIQUETAS DE SALTO
-------------------------------------------------
  while_start_0                se resuelve en la otra etapa
  while_end_1                  se resuelve en la otra etapa
  else_2                       se resuelve en la otra etapa
  endif_3                      se resuelve en la otra etapa
  else_4                       se resuelve en la otra etapa
  endif_5                      se resuelve en la otra etapa


                    MAPA DE MEMORIA
  NOMBRE              TIPO              DIRECCION    AMBITO
-------------------------------------------------------------
  datos               [5]int            0x0000   local (buscar)
  objetivo            int               0x0014   local (buscar)
  i                   int               0x0018   local (buscar)
  encontrado          int               0x001C   local (buscar)
  posicion            int               0x0020   local (buscar)
  datos               [5]int            0x0000   local (main)
  objetivo            int               0x0014   local (main)
  pos                 int               0x0018   local (main)
```

```
=== ASM GENERADO ===

lli r2, 0, 0
lli r2, 0xE0, 1

# --- inicialización de variables globales ---:

jal r1, main
halt

# --- funciones ---:
buscar:
addi r2, r2, -4
addi r8, r0, 0
store r8, 0(r2)
addi r2, r2, -4
addi r8, r0, 0
store r8, 0(r2)
addi r2, r2, -4
addi r8, r0, 0
store r8, 0(r2)
while_start_0:
load r8, 8(r2)
addi r9, r0, 5
bge r8, r9, while_end_1
addi r9, r0, 0
load r8, 8(r2)
addi r10, r0, 2
sll r11, r8, r10
add r8, r9, r11
load r9, 0(r8)
add r8, r5, r0
bne r9, r8, else_2
addi r8, r0, 1
store r8, 4(r2)
load r8, 8(r2)
store r8, 0(r2)
addi r8, r0, 5
store r8, 8(r2)
jal r0, endif_3
else_2:
load r8, 8(r2)
addi r9, r0, 1
add  r11, r8, r9
store r11, 8(r2)
endif_3:
jal r0, while_start_0
while_end_1:
load r11, 4(r2)
```

```
=== ASM RESUELTO ===

lli r2, 0, 0
lli r2, 0xE0, 1
jal r1, 48
halt
addi r2, r2, -4
addi r8, r0, 0
store r8, 0(r2)
addi r2, r2, -4
addi r8, r0, 0
store r8, 0(r2)
addi r2, r2, -4
addi r8, r0, 0
store r8, 0(r2)
load r8, 8(r2)
addi r9, r0, 5
bge r8, r9, 21
addi r9, r0, 0
load r8, 8(r2)
addi r10, r0, 2
sll r11, r8, r10
add r8, r9, r11
load r9, 0(r8)
add r8, r5, r0
bne r9, r8, 8
addi r8, r0, 1
store r8, 4(r2)
load r8, 8(r2)
store r8, 0(r2)
addi r8, r0, 5
store r8, 8(r2)
jal r0, 5
load r8, 8(r2)
addi r9, r0, 1
add  r11, r8, r9
store r11, 8(r2)
jal r0, -22
load r11, 4(r2)
addi r9, r0, 1
bne r11, r9, 6
load r9, 0(r2)
add r4, r9, r0
addi r2, r2, 12
jr r1
jal r0, 5
addi r9, r0, 0
```

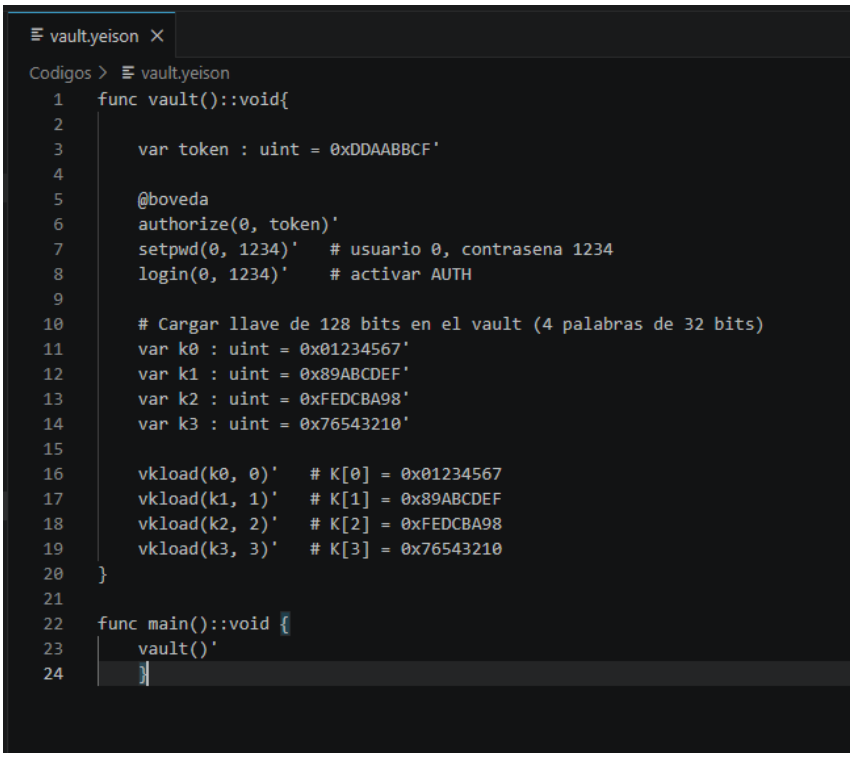
```
=== TABLA DE ETIQUETAS (Resolver) ===
  buscar                          0x0010
  while_start_0                   0x0034
  else_2                          0x007C
  endif_3                         0x008C
  while_end_1                     0x0090
  else_4                          0x00B0
  endif_5                         0x00C0
  main                            0x00C8
Binario generado : .\Codigos\busqueda.bin  (80 instrucciones, 320 bytes)
Hex dump generado: .\Codigos\busqueda.mem
```

busqueda.mem

Codigos > busqueda.mem

```
110000 // lli r2, 0, 0
1102E0 // lli r2, 0xE0, 1
388030 // jal r1, 48
480000 // halt
0917FC // addi r2, r2, -4
0C0000 // addi r8, r0, 0
241000 // store r8, 0(r2)
0917FC // addi r2, r2, -4
0C0000 // addi r8, r0, 0
241000 // store r8, 0(r2)
0917FC // addi r2, r2, -4
0C0000 // addi r8, r0, 0
241000 // store r8, 0(r2)
1C1008 // load r8, 8(r2)
0C8005 // addi r9, r0, 5
34C015 // bge r8, r9, 21
0C8000 // addi r9, r0, 0
1C1008 // load r8, 8(r2)
0D0002 // addi r10, r0, 2
65C550 // sll r11, r8, r10
644D80 // add r8, r9, r11
1CC000 // load r9, 0(r8)
642800 // add r8, r5, r0
2C4C08 // bne r9, r8, 8
0C0001 // addi r8, r0, 1
241004 // store r8, 4(r2)
1C1008 // load r8, 8(r2)
241000 // store r8, 0(r2)
0C0005 // addi r8, r0, 5
241008 // store r8, 8(r2)
380005 // jal r0, 5
1C1008 // load r8, 8(r2)
0C8001 // addi r9, r0, 1
65C480 // add  r11, r8, r9
259008 // store r11, 8(r2)
387FEA // jal r0, -22
1D9004 // load r11, 4(r2)
0C8001 // addi r9, r0, 1
2CDC06 // bne r11, r9, 6
1C9000 // load r9, 0(r2)
624800 // add r4, r9, r0
```

Ejercicio 3. Vault



```
piler-feature-bin\custom-binary-compiler-feature-bin\main.py .\Codigos\vault.yei
--------------------------------------------------------------------------
                              TOKENS
--------------------------------------------------------------------------
  #     TIPO                VALOR                LIN:COL
  ----------------------------------------------------------------------
  0     FUNC                'func'               1:1
  1     IDENTIFIER          'vault'              1:6
  2     LPAREN              '('                  1:11
  3     RPAREN              ')'                  1:12
  4     OP_RETURN_TYPE      '::'                 1:13
  5     VOID                'void'               1:15
  6     LBRACE              '{'                  1:19
  7     VAR                 'var'                3:5
  8     IDENTIFIER          'token'              3:9
  9     COLON               ':'                  3:15
  10    UINT                'uint'               3:17
  11    ASSIGN              '='                  3:22
  12    HEXADECIMAL         3718953935           3:24
  13    TERMINATOR          "'"                  3:34
  14    ANNOTATION          '@boveda'            5:5
  15    AUTHORIZE           'authorize'          6:5
  16    LPAREN              '('                  6:14
  17    INTEGER             0                    6:15
  18    COMMA               ','                  6:16
  19    IDENTIFIER          'token'              6:18
  20    RPAREN              ')'                  6:23
  21    TERMINATOR          "'"                  6:24
  22    SETPWD              'setpwd'             7:5
  23    LPAREN              '('                  7:11
  24    INTEGER             0                    7:12
  25    COMMA               ','                  7:13
  26    INTEGER             1234                 7:15
  27    RPAREN              ')'                  7:19
  28    TERMINATOR          "'"                  7:20
  29    LOGIN               'login'              8:5
  30    LPAREN              '('                  8:10
  31    INTEGER             0                    8:11
  32    COMMA               ','                  8:12
  33    INTEGER             1234                 8:14
  34    RPAREN              ')'                  8:18
  35    TERMINATOR          "'"                  8:19
  36    VAR                 'var'                11:5
  37    IDENTIFIER          'k0'                 11:9
  38    COLON               ':'                  11:12
  39    UINT                'uint'               11:14
  40    ASSIGN              '='                  11:19
  41    HEXADECIMAL         19088743             11:21
  42    TERMINATOR          "'"                  11:31
  43    VAR                 'var'                12:5
```

```
=== ÁRBOL SINTÁCTICO ===

└── Program
    └── SectionBlock(@code)
        ├── FunctionDeclaration(vault)
        │   ├── Type(void)
        │   ├── VarDeclaration(token)
        │   │   ├── Type(uint)
        │   │   └── HexLiteral(0xDDAABBCF)
        │   └── SectionBlock(@boveda)
        │       ├── ExpressionStatement
        │       │   └── FunctionCall(authorize)
        │       │       ├── IntLiteral(0)
        │       │       └── Identifier(token)
        │       ├── ExpressionStatement
        │       │   └── FunctionCall(setpwd)
        │       │       ├── IntLiteral(0)
        │       │       └── IntLiteral(1234)
        │       ├── ExpressionStatement
        │       │   └── FunctionCall(login)
        │       │       ├── IntLiteral(0)
        │       │       └── IntLiteral(1234)
        │       ├── VarDeclaration(k0)
        │       │   ├── Type(uint)
        │       │   └── HexLiteral(0x1234567)
        │       ├── VarDeclaration(k1)
        │       │   ├── Type(uint)
        │       │   └── HexLiteral(0x89ABCDEF)
        │       ├── VarDeclaration(k2)
        │       │   ├── Type(uint)
        │       │   └── HexLiteral(0xFEDCBA98)
        │       ├── VarDeclaration(k3)
        │       │   ├── Type(uint)
        │       │   └── HexLiteral(0x76543210)
        │       ├── ExpressionStatement
        │       │   └── FunctionCall(vkload)
        │       │       ├── Identifier(k0)
        │       │       └── IntLiteral(0)
        │       ├── ExpressionStatement
        │       │   └── FunctionCall(vkload)
        │       │       ├── Identifier(k1)
        │       │       └── IntLiteral(1)
        │       ├── ExpressionStatement
        │       │   └── FunctionCall(vkload)
        │       │       ├── Identifier(k2)
        │       │       └── IntLiteral(2)
        │       └── ExpressionStatement
        │           └── FunctionCall(vkload)
```

```
piler-feature-bin\custom-binary-compiler-feature-bin\main.py .\Codigos\vault.yeison -a
TABLA DE SIMBOLOS
------------------------------------------------------------------------
  NOMBRE               KIND     TIPO               AMBITO          DIRECCION

ambito: global
  vault                func     () -> void         global          N/A
  main                 func     () -> void         global          N/A

ambito: vault (local)
  token                var      uint               vault           0x0000
  k0                   var      uint               vault           0x0004
  k1                   var      uint               vault           0x0008
  k2                   var      uint               vault           0x000C
  k3                   var      uint               vault           0x0010


               ETIQUETAS DE SALTO
------------------------------------------------


                   MAPA DE MEMORIA
  NOMBRE               TIPO               DIRECCION    AMBITO
-------------------------------------------------------------
  token                uint               0x0000   local (vault)
  k0                   uint               0x0004   local (vault)
  k1                   uint               0x0008   local (vault)
  k2                   uint               0x000C   local (vault)
  k3                   uint               0x0010   local (vault)
```

```
piler-feature-bin\custom-binary-compiler-feature-bin\main.py .\Codigos\vault.yeison -a
=== ASM GENERADO ===

lli r2, 0, 0
lli r2, 0xE0, 1

# --- inicialización de variables globales ---:

jal r1, main
halt

# --- funciones ---:
vault:
addi r2, r2, -4
lli r8, 207, 0
lli r8, 187, 1
lli r8, 170, 2
lli r8, 221, 3
store r8, 0(r2)
addi r8, r0, 0
load r9, 0(r2)
authorize r8, r9
addi r9, r0, 0
lli r8, 210, 0
lli r8, 4, 1
setpwd r9, r8
addi r8, r0, 0
lli r9, 210, 0
lli r9, 4, 1
login r8, r9
addi r2, r2, -4
lli r9, 103, 0
lli r9, 69, 1
lli r9, 35, 2
lli r9, 1, 3
store r9, 0(r2)
addi r2, r2, -4
lli r9, 239, 0
lli r9, 205, 1
lli r9, 171, 2
lli r9, 137, 3
store r9, 0(r2)
addi r2, r2, -4
lli r9, 152, 0
lli r9, 186, 1
lli r9, 220, 2
lli r9, 254, 3
store r9, 0(r2)
addi r2, r2, -4
lli r9, 16, 0
```

```
=== ASM RESUELTO ===

lli r2, 0, 0
lli r2, 0xE0, 1
jal r1, 53
halt
addi r2, r2, -4
lli r8, 207, 0
lli r8, 187, 1
lli r8, 170, 2
lli r8, 221, 3
store r8, 0(r2)
addi r8, r0, 0
load r9, 0(r2)
authorize r8, r9
addi r9, r0, 0
lli r8, 210, 0
lli r8, 4, 1
setpwd r9, r8
addi r8, r0, 0
lli r9, 210, 0
lli r9, 4, 1
login r8, r9
addi r2, r2, -4
lli r9, 103, 0
lli r9, 69, 1
lli r9, 35, 2
lli r9, 1, 3
store r9, 0(r2)
addi r2, r2, -4
lli r9, 239, 0
lli r9, 205, 1
lli r9, 171, 2
lli r9, 137, 3
store r9, 0(r2)
addi r2, r2, -4
lli r9, 152, 0
lli r9, 186, 1
lli r9, 220, 2
lli r9, 254, 3
store r9, 0(r2)
addi r2, r2, -4
lli r9, 16, 0
lli r9, 50, 1
lli r9, 84, 2
lli r9, 118, 3
store r9, 0(r2)
load r9, 12(r2)
```

```
=== TABLA DE ETIQUETAS (Resolver) ===
  vault                          0x0010
  main                           0x00DC
Binario generado : .\Codigos\vault.bin  (62 instrucciones, 248 bytes)
Hex dump generado: .\Codigos\vault.mem
```

vault.mem

Codigos > vault.mem

```
110000 // lli r2, 0, 0
1102E0 // lli r2, 0xE0, 1
388035 // jal r1, 53
480000 // halt
0917FC // addi r2, r2, -4
1400CF // lli r8, 207, 0
1402BB // lli r8, 187, 1
1404AA // lli r8, 170, 2
1406DD // lli r8, 221, 3
241000 // store r8, 0(r2)
0C0000 // addi r8, r0, 0
1C9000 // load r9, 0(r2)
6C4803 // authorize r8, r9
0C8000 // addi r9, r0, 0
1400D2 // lli r8, 210, 0
140204 // lli r8, 4, 1
6CC000 // setpwd r9, r8
0C0000 // addi r8, r0, 0
1480D2 // lli r9, 210, 0
148204 // lli r9, 4, 1
6C4801 // login r8, r9
0917FC // addi r2, r2, -4
148067 // lli r9, 103, 0
148245 // lli r9, 69, 1
148423 // lli r9, 35, 2
148601 // lli r9, 1, 3
249000 // store r9, 0(r2)
0917FC // addi r2, r2, -4
1480EF // lli r9, 239, 0
1482CD // lli r9, 205, 1
1484AB // lli r9, 171, 2
148689 // lli r9, 137, 3
249000 // store r9, 0(r2)
0917FC // addi r2, r2, -4
148098 // lli r9, 152, 0
1482BA // lli r9, 186, 1
1484DC // lli r9, 220, 2
1486FE // lli r9, 254, 3
249000 // store r9, 0(r2)
0917FC // addi r2, r2, -4
148010 // lli r9, 16, 0
148232 // lli r9, 50, 1
148454 // lli r9, 84, 2
148676 // lli r9, 118, 3
249000 // store r9, 0(r2)
1C900C // load r9, 12(r2)
684804 // vkload r9, 0
1C9008 // load r9, 8(r2)
```